

LU3IN017 : RAPPORT - Projet Organiz'Asso par Alexandra Ioana-Bolea & Bettina Mubiligi

LU3IN017 : RAPPORT - Projet Organiz'Asso par Alexandra Ioana-Bolea & Bettina Mubiligi.....	1
Limites de notre implémentation (ce qui reste à faire).....	2
Difficultés rencontrées.....	2
Répartition du travail.....	2
Arborescence du projet + Comment lancer le forum.....	3
Schéma de la BDD.....	4
Sources et Bibliographie.....	5
Exemple de jeu de tests - requêtes CURL (également disponible dans un fichier test du zip).....	6

Limites de notre implémentation (ce qui reste à faire)

- Un admin qui a déjà un compte admin doit avoir un compte membre pour parler aux membres (les admins peuvent voir tous les messages de tous les forums (ouvert ou fermé) mais les membres ne voient que les messages du forum ouvert, donc il ne voient pas les messages que les admins s'envoient entre eux. Donc un membre ne peut pas voir un message d'un admin, d'où la nécessité d'un 2e compte (membre) pour les admins.
- Pas de nom/prénom dans les identifiants mais un login seulement par l'adresse mail/password (plus facile pour nous et pour la gestion de la bdd)
- Connexions simultanées sur plusieurs pages difficiles (déconnexion, bugs, etc).
- Pas de modification de messages possible (nécessiterait une route PUT/PATCH pour modifier un message dans api.js)
- La fonctionnalité de droits d'admin du sujet n'a pas été implémentée par oubli (et manque de temps car on s'en est rendues compte à la dernière minute)
- Quelques bugs (se reconnecter en tant qu'admin si la première connexion n'amène pas à la page admin complète avec le champs de demande d'inscriptions)
- Compte(s) admin(s) créé.s directement via la BDD (car impossible de valider leur inscription sinon – pas de possibilité de transformer quelqu'un en admin (point mentionné plus haut.)

Difficultés rencontrées

Les difficultés rencontrées principalement étaient la liaison entre composants : comprendre comment lier le frontend React au backend avec le serveur et la BDD. De plus, en cas d'erreur à la modification d'un composant React, plus rien ne marchait et il fallait systématiquement déboguer doucement pour comprendre nos erreurs. Nous avons également eu des problèmes dans la compréhension de la gestion des ports (serveur qui écoute sur 3001, React sur 5173 et Bdd sur 2701 etc.), et dans la gestion de l'arborescence des fichiers pour ne pas se perdre (plusieurs fichiers package.json, etc). Le lancement du serveur en lui-même aussi était complexe, avec la gestion des fichiers index.js et app.js. Et bien sûr la gestion du temps et du travail en binôme pour un projet comme celui-ci.

Répartition du travail

Globalement, nous avons chacune fait 50% du travail, notamment au début (frontend) où l'une s'est chargée des pages React de connexion/enregistrement pendant que l'autre se chargeait des pages principales pour les forums ouverts/fermés. Pour le backend, nous avons toutes les deux travaillé ensemble sur la partie server (l'une créait les routes/requêtes pour le login pendant que l'autre créait celle du log out etc), et nous avons fini l'implémentation comme cela, avec des séances en présentiel et en distanciel.

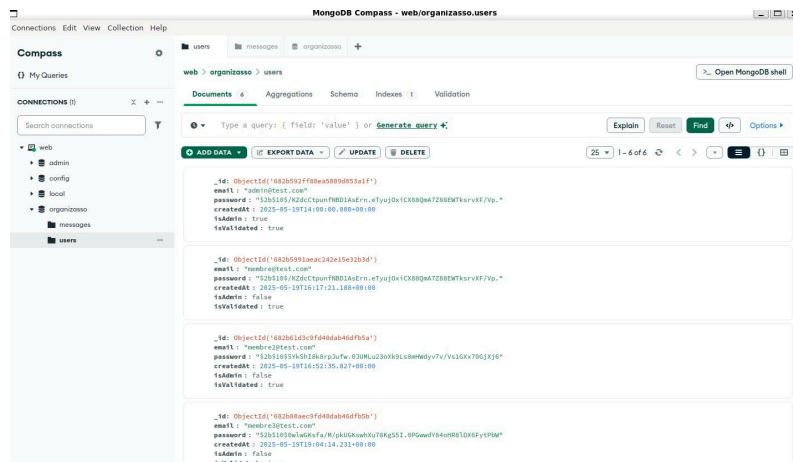
Arborescence du projet + Comment lancer le forum

Ouvrir un terminal avec 3 pages :

- Page 1 : on entre dans Projet/src et on lance npm run dev pour lancer le front end avec Vite+react
- Page 2 : on lance le server avec la BDD en backend avec npm start (messages de connexion si tout se passe bien)
- Page 3 (Optionnel) : on lance mongodb-compass pour voir l'interface graphique de la BDD et observer/modifier la BDD à sa guise

Note : utilisation partielle de l'arborescence du projet du TD/TME7 avec quelques différences : les fichiers .JSX sont toujours dans le même répertoire que leurs fichiers .CSS correspondants. Le côté client est contenu dans le répertoire src principal, avec ses fichiers .JSON, .JSX et .HTML. Le côté serveur est dans le répertoire src/server/, avec ses fichiers à lui.

Schéma de la BDD



Création d'une base de données "organizasso" avec deux collections :

- "users" : informations sur les utilisateurs, avec les entrées suivantes
_id : ObjectId
email : String
Password : String
createdAt : Date
isAdmin : Boolean - automatiquement à false pour tous les nouveaux users
isValidated : Boolean - automatiquement à "false" pour tous les nouveaux users, change en "true" si validation par un admin.

- "messages" : stockage des messages, de la forme :

```
{
  "_id": {
    "$oid": "682dc2cb370824b948ea3aac"
  },
  "content": "Ceci est un message privé",
  "authorEmail": "admin@test.com",
  "authorRole": "admin",
  "forumType": "closed",
  "date": {
    "$date": "2025-05-21T12:10:51.159Z"
  }
}
```

}

Sources et Bibliographie

React

<https://fr.react.dev/reference/react/hooks#state-hooks>

<https://www.w3schools.com/REACT/DEFAULT.ASP>

<https://fuel.digital/work/make-a-react-website-responsive-step-by-step/>

Node js, express et MongoDB + compass

<https://openclassrooms.com/fr/courses/6390246-passez-au-full-stack-avec-node-js-express-et-mongodb#table-of-content>

<https://docs.npmjs.com/cli/v11/configuring-npm/package-json>

<https://welovedevs.com/fr/articles/nodejs-mongodb/>

<https://welovedevs.com/fr/articles/api-node-js-mongodb/>

https://developer.mozilla.org/fr/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs/Introduction

<https://serverspace.io/fr/support/help/how-to-use-mongodb-compass/>

Axios et Javascript

<https://axios-http.com/fr/docs/intro>

<https://v2.fr.vuejs.org/v2/cookbook/using-axios-to-consume-apis.html>

https://www.w3schools.com/js/js_syntax.asp

Tests avec CURL

<https://terminalcheatsheet.com/guides/curl-rest-api>

Chiffrement/hachage avec bcrypt

<https://laconsole.dev/blog/hacher-mot-de-passe-js-bcrypt>

<https://www.freecodecamp.org/news/how-to-hash-passwords-with-bcrypt-in-nodejs/#heading-how-to-install-bcrypt-in-nodejs>

+++ documentation CSS, React, cours fournis et utilisation de l'IA pour déboguer notre code en cas de problème.

Exemple de jeu de tests - requêtes CURL (également disponible dans un fichier test du zip)

Dans ce jeu de test, on utilise l'environnement que l'on a créer (le serveur écoute le port 3001, le site se triuve sur le port `http://127.0.0.1:5173/` du localhost et la BDD MongoDB sur `mongodb://localhost:27017`)

1. Créer un utilisateur (inscription)

```
curl -X POST http://localhost:3001/api/users/register \  
-H "Content-Type: application/json" \  
-d '{"email":"testuser@example.com","password":"motdepasse"}'
```

2. pour se connecter (login)

```
curl -X POST http://localhost:3001/api/users/login \  
-H "Content-Type: application/json" \  
-d '{"email":"testuser@example.com","password":"motdepasse"}'
```

3. pour poster un message dans le forum ouvert

```
curl -X POST http://localhost:3001/api/messages \  
-H "Content-Type: application/json" \  
-d '{"content":"Ceci est un message  
public","authorEmail":"testuser@example.com","authorRole":"Membre","forumType":"open"}'
```

4. poster un message dans le forum fermé (admin)

```
curl -X POST http://localhost:3001/api/messages \  
-H "Content-Type: application/json" \  
-d '{"content":"Ceci est un message  
privé","authorEmail":"admin@example.com","authorRole":"admin","forumType":"closed"}'
```

5. Lister tous les messages du forum ouvert

curl http://localhost:3001/api/messages?forumType=open

6. Lister tous les messages du forum ferme (admin)

curl http://localhost:3001/api/messages?forumType=closed

7. pour tous les messages (admin)

curl http://localhost:3001/api/messages

8. supprimer un message (il faut remplacer MESSAGE_ID et email)

curl -X DELETE

"http://localhost:3001/api/messages/MESSAGE_ID?authorEmail=testuser@example.com"

9. et lister les demandes d'inscription en attente (admin)

curl http://localhost:3001/api/admin/pending-users

11. recherche des messages contenant un mot clé (filtrage)

curl http://localhost:3001/api/messages?forumType=open | grep "motclé"